

# Data Types, Variables, Operators

## Study Guide

**Mr. Shivkumar Lilhare**  
Assistant Professor(CSE), PIT  
Parul University

|           |                                                   |          |
|-----------|---------------------------------------------------|----------|
| <b>1.</b> | <b>Introduction to Operators.....</b>             | <b>1</b> |
| <b>2.</b> | <b>Operator Precedence and Associativity.....</b> | <b>5</b> |
| <b>3.</b> | <b>Summary of Key Concepts.....</b>               | <b>6</b> |
| <b>4.</b> | <b>References.....</b>                            | <b>6</b> |

## 2.4.1 Introduction to Operators

**Definition:** Operators are symbols that perform operations on variables and values. Java offers a variety of operators used in expressions and control statements.

### ➤ Types of Operators

#### 1. Arithmetic Operators

Used to perform basic mathematical operations.

| Operator | Operation      | Example | Result |
|----------|----------------|---------|--------|
| +        | Addition       | 5 + 3   | 8      |
| -        | Subtraction    | 5 - 3   | 2      |
| *        | Multiplication | 5 * 3   | 15     |
| /        | Division       | 10 / 2  | 5      |
| %        | Modulo         | 10 % 3  | 1      |

**Notes:**

- Integer division truncates the decimal:  $7 / 2 = 3$
- Modulus is often used to determine even/odd:  $x \% 2 == 0$

#### 2. Relational (Comparison) Operators

Used to compare two values.

| Operator | Meaning          | Example  | Output             |
|----------|------------------|----------|--------------------|
| ==       | Equal to         | $x == y$ | true if equal      |
| !=       | Not equal to     | $x != y$ | true if not equal  |
| >        | Greater than     | $x > y$  | true if $x > y$    |
| <        | Less than        | $x < y$  | true if $x < y$    |
| >=       | Greater or equal | $x >= y$ | true if $x \geq y$ |
| <=       | Less or equal    | $x <= y$ | true if $x \leq y$ |

### 3. Logical Operators

Used to combine multiple conditions, especially in control flow (if, while).

| Operator | Description | Example          | Result                                |
|----------|-------------|------------------|---------------------------------------|
| &&       | AND         | (x > 0 && y > 0) | true only if both are true            |
|          | OR          | (x > 0    y > 0) | True if any condition is true or both |
| !        | NOT         | !(x > 0)         | Inverts result                        |

**Example:**

```
int age = 25;
if (age > 18 && age < 60) {
    System.out.println("Working age");
}
```

### 4. Bitwise Operators

Operate on individual bits of integer values.

| Operator | Meaning            | Example (x=5, y=3) | Binary Operation   | Result        |
|----------|--------------------|--------------------|--------------------|---------------|
| &        | Bitwise AND        | x & y              | 0101 & 0011 → 0001 | 1             |
|          |                    | x   y              | 0101   0011 → 0111 | 7             |
| ^        | Bitwise XOR        | x ^ y              | 0101 ^ 0011 → 0110 | 6             |
| ~        | Bitwise Complement | ~x                 | ~0101 → 1010       | -6 (2's comp) |
| <<       | Left shift         | x << 1             | 0101 << 1 → 1010   | 10            |
| >>       | Right shift        | x >> 1             | 0101 >> 1 → 0010   | 2             |

## 5. Assignment Operators

Assign values to variables and modify existing values.

| Operator | Meaning             | Equivalent to                  |
|----------|---------------------|--------------------------------|
| =        | Assignment          | $x = y$                        |
| +=       | Add and assign      | $x += 5 \rightarrow x = x + 5$ |
| -=       | Subtract and assign | $x -= 3$                       |
| *=       | Multiply and assign | $x *= 2$                       |
| /=       | Divide and assign   | $x /= 4$                       |
| %=       | Modulus and assign  | $x \% = 2$                     |

**Example:**

```
int x = 10;
x += 5; // x = x + 5 = 15
```

## 6. Conditional (Ternary) Operator

Used for shorthand if-else.

**Syntax:**

Condition ? expression1 : expression2;

**Example:**

```
int a = 5, b = 10;
int max = (a > b) ? a : b; // max = 10
```

## 7. instanceof Operator

Used to test whether an object is an instance of a specific class or subclass.

**Example:**

```
if (obj instanceof String) { ... }
```

## 8. Unary Operators in Java

### Definition:

Unary operators are those that operate on **a single operand** to perform various operations like incrementing/decrementing, negating a value, or inverting a boolean value.

| Operator | Description                      | Example    | Result         |
|----------|----------------------------------|------------|----------------|
| +        | Unary plus (positive value)      | +a         | Returns a      |
| -        | Unary minus (negation)           | -a         | Negates a      |
| ++       | Increment operator               | ++a or a++ | Adds 1 to a    |
| --       | Decrement operator               | --a or a-- | Subtracts 1    |
| !        | Logical NOT (boolean complement) | !flag      | Reverses value |

### Examples:

```
int x = 10;

System.out.println(+x); // 10
System.out.println(-x); // -10

boolean flag = false;

System.out.println(!flag); // true

int a = 5;

System.out.println(a++); // 5 (then a becomes 6)
System.out.println(++a); // 7
```

## 2.4.2 Operator Precedence and Associativity

- **Operator Precedence** determines **which operator is evaluated first** when an expression has more than one operator.
- **Associativity** determines the **order of evaluation** when operators have the **same precedence** (i.e., left-to-right or right-to-left).

### ✓ Operator Precedence and Associativity Table

| Precedence Level | Operators                           | Associativity | Description                              |
|------------------|-------------------------------------|---------------|------------------------------------------|
| 1 (Highest)      | (), [], .                           | Left to Right | Method call, array access, member access |
| 2                | ++, --, +, -, ~, !                  | Right to Left | Unary operators                          |
| 3                | *, /, %                             | Left to Right | Multiplication, Division, Modulus        |
| 4                | +, -                                | Left to Right | Addition, Subtraction                    |
| 5                | <<, >>, >>>                         | Left to Right | Bitwise shift operators                  |
| 6                | <, <=, >, >=, instanceof            | Left to Right | Relational operators                     |
| 7                | ==, !=                              | Left to Right | Equality operators                       |
| 8                | &                                   | Left to Right | Bitwise AND                              |
| 9                | ^                                   | Left to Right | Bitwise XOR                              |
| 10               |                                     | Left to Right | Bitwise OR                               |
| 11               | &&                                  | Left to Right | Logical AND                              |
| 12               |                                     | Left to Right | Logical OR                               |
| 13               | ?:                                  | Right to Left | Ternary (conditional)                    |
| 14               | =, +=, -=, *=, /=, %=, &=,  =, etc. | Right to Left | Assignment operators                     |
| 15 (Lowest)      | ,                                   | Left to Right | Comma (used to separate expressions)     |

## ➤ Summary of Key Concepts:

Java provides various operators to perform computations and logic:

- Arithmetic Operators (+, -, \*, /, %) perform basic math.
- Relational Operators (==, !=, >, <, >=, <=) compare values.
- Logical Operators (&&, ||, !) combine or invert boolean expressions.
- Bitwise Operators (&, |, ^, ~, <<, >>, >>>) work at the bit level.
- Assignment Operators (=, +=, -=, etc.) assign values to variables.
- Conditional Operator (?) is a shorthand for if-else.

Operator precedence determines evaluation order, and associativity resolves tie-breaking. Operators are widely used in expressions to control program logic and calculations.

## ➤ References:

- *Java: The Complete Reference* by Herbert Schildt
- *Head First Java* by Kathy Sierra & Bert Bates
- Oracle Official Documentation: [docs.oracle.com/javase](https://docs.oracle.com/javase)
- GeeksforGeeks: [www.geeksforgeeks.org](https://www.geeksforgeeks.org)
- TutorialsPoint: [www.tutorialspoint.com/java](https://www.tutorialspoint.com/java)



**Parul<sup>®</sup> University** | **NAAC** **A++**  
Vadodara, Gujarat | **GRADE**



      
<https://paruluniversity.ac.in/>